

Chapter 21

Uncertainty Propagation

Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Slides

Chapter Overview

- 1 Introduction
- 2 21.1 Sampling Methods (Monte Carlo)
- 3 21.2 Taylor Approximation
- 4 21.3 Polynomial Chaos
- 5 21.4 Bayesian Monte Carlo
- 6 Method Comparison
- 7 21.5 Summary & Exercises

Motivation: Uncertainty in Optimization

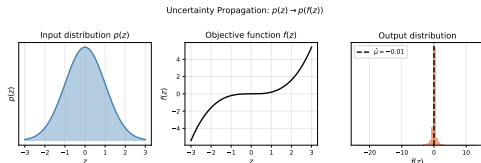
Problem Setting

In probabilistic optimization under uncertainty, some inputs to the objective $f(x, z)$ are modelled as a **random variable** $z \sim p(z)$ over domain $\mathcal{Z}$.

Goal: propagate the known input distribution $p(z)$ to estimate quantities of the *output* distribution:

- Expected value $\mathbb{E}_{z \sim p}[f(z)]$
- Variance $\text{Var}_{z \sim p}[f(z)]$

Note: for brevity we drop x from $f(x, z)$; the dependency still exists and we recompute per design point.



Four Approaches

- 1 **Monte Carlo Sampling**
No derivatives; black-box; slow convergence
- 2 **Taylor Approximation**
Requires derivatives; analytic; fast
- 3 **Polynomial Chaos**
Surrogate model; orthogonal basis fit
- 4 **Bayesian Monte Carlo**
GP surrogate; analytic for Gaussian inputs

Key quantities

$f(z)$: objective function
 $p(z)$: known input distribution
 $\hat{\mu}$: estimated mean
 $\hat{\nu}$: estimated variance

Trade-offs

Accuracy vs. computational cost
function evaluations needed
Assumptions on f smoothness

21.1 Monte Carlo Sampling

Sample Mean and Sample Variance (Eqs. 21.1–21.2)

Draw m i.i.d. samples $z^{(1)}, \dots, z^{(m)} \sim p(z)$:

$$\mathbb{E}_{z \sim p}[f(z)] \approx \hat{\mu} = \frac{1}{m} \sum_{i=1}^m f(z^{(i)})$$

$$\text{Var}_{z \sim p}[f(z)] \approx \hat{\nu} = \left(\frac{1}{m} \sum_{i=1}^m f(z^{(i)})^2 \right) - \hat{\mu}^2$$

m number of samples

$z^{(i)}$ i -th sample from p

$\hat{\mu}$ sample mean (estimator of $\mathbb{E}[f]$)

$\hat{\nu}$ sample variance (estimator of $\text{Var}[f]$)

Monte Carlo: Convergence Properties

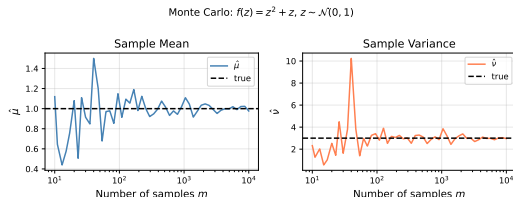
Variance of the estimator

For normally distributed f :

$$\text{Var}[\hat{\mu}] = \frac{\nu}{m}$$

Doubling m *halves* the variance.

Convergence rate: $\mathcal{O}(m^{-1/2})$



$$f(z) = z^2 + z, z \sim \mathcal{N}(0, 1)$$

Advantages:

- p need not be known exactly
- Works with simulation / real experiments
- No smoothness assumptions on f

Limitation:

- Slow convergence; many evaluations needed
- Quasi-MC (Ch. 16) gives faster convergence

Monte Carlo in Python

```
1 import numpy as np
2
3 def monte_carlo_moments(f, sample_fn, m=10000, seed=42):
4     """
5     Estimate mean and variance of  $f(z)$  using Monte Carlo.
6     Parameters
7     -----
8     f           : callable, objective function  $f(z)$ 
9     sample_fn   : callable, draws 1 sample from  $p(z)$ 
10    m            : int, number of samples
11    Returns
12    -----
13    mu_hat, nu_hat : float, estimated mean and variance
14    """
15    rng = np.random.default_rng(seed)
16    samples = np.array([f(sample_fn(rng)) for _ in range(m)])
17    mu_hat = np.mean(samples)
18    nu_hat = np.mean(samples**2) - mu_hat**2
19    return mu_hat, nu_hat
20
21 # Example:  $f(z) = \sin(z)$ ,  $z \sim N(0,1)$ 
22 f = np.sin
23 sample_fn = lambda rng: rng.standard_normal()
24
25 mu, nu = monte_carlo_moments(f, sample_fn, m=5000)
26 print(f"mu_hat = {mu:.4f}")      # -> ~0.0
27 print(f"nu_hat = {nu:.4f}")      # -> ~0.4597
```

21.2 Taylor Approximation — Setup

Second-Order Taylor Expansion of $f(\mathbf{z})$ at $\mathbf{z} = \boldsymbol{\mu}_z$

$$\hat{f}(\mathbf{z}) = f(\boldsymbol{\mu}_z) + \sum_{i=1}^n \frac{\partial f}{\partial z_i} (z_i - \mu_i) + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 f}{\partial z_i \partial z_j} (z_i - \mu_i)(z_j - \mu_j)$$

Assumptions:

- Components of $\mathbf{z}$ are **uncorrelated** (or whitened)
- Each z_i has mean μ_i and variance ν_i
- All derivatives evaluated at $\mathbf{z} = \boldsymbol{\mu}_z$

Whitening correlated inputs

If $\mathbf{z} = \mathbf{c}$ with covariance $\mathbf{C}$, transform: $\mathbf{z} = \mathbf{T}\mathbf{c}$, where $\mathbf{T}$ contains the eigenvectors of $\mathbf{C}$ (columns ordered by the m largest eigenvalues).

Taylor Approximation — Mean and Variance Formulas

Second-Order Estimates (Eqs. 21.4–21.5)

$$\hat{\mu} = f(\boldsymbol{\mu}_z) + \frac{1}{2} \sum_{i=1}^n \frac{\partial^2 f}{\partial z_i^2} \nu_i \bigg|_{\mathbf{z}=\boldsymbol{\mu}_z}$$
$$\hat{\nu} = \sum_{i=1}^n \left(\frac{\partial f}{\partial z_i} \right)^2 \nu_i + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \left(\frac{\partial^2 f}{\partial z_i \partial z_j} \right)^2 \nu_i \nu_j \bigg|_{\mathbf{z}=\boldsymbol{\mu}_z}$$

First-Order Approximation (Eq. 21.6) — Simpler

$$\hat{\mu} \approx f(\boldsymbol{\mu}_z), \quad \hat{\nu} \approx \sum_{i=1}^n \left(\frac{\partial f}{\partial z_i} \right)^2 \nu_i \bigg|_{\mathbf{z}=\boldsymbol{\mu}_z}$$

Neglects higher-order terms; efficient when f is nearly linear.

Symbol guide: $\nu_i = \text{Var}[z_i]$, $\boldsymbol{\mu}_z = \mathbb{E}[\mathbf{z}]$, $n = \text{dimension of } \mathbf{z}$.

Taylor Approximation — Numerical Example

Example 21.1: $f(x, z) = \sin(x + z_1) \cos(x + z_2)$

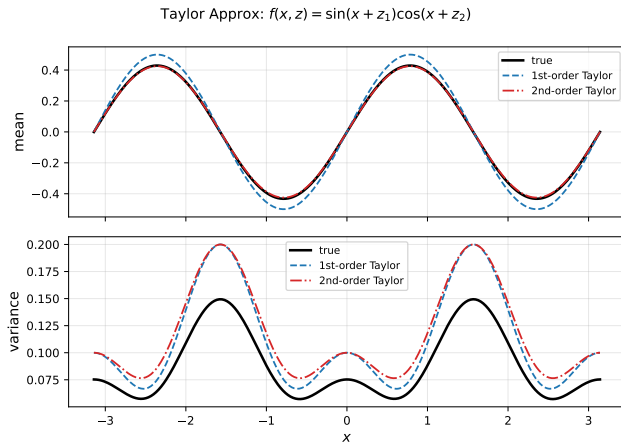
$z_1 \sim \mathcal{N}(0, 0.1)$, $z_2 \sim \mathcal{N}(0, 0.2)$; evaluate at $\mu_z = (0, 0)$.

Partial derivatives at $z = 0$: $\frac{\partial f}{\partial z_1} = \cos(x) \cos(x)$, $\frac{\partial f}{\partial z_2} = -\sin(x) \sin(x)$

Resulting approximations:

$$\hat{\mu}(x) = 0.85 \sin(x) \cos(x), \quad \hat{\nu}(x) = 0.1 \cos^4(x) + 0.2 \sin^4(x) + 0.045 \sin^2(x) \cos^2(x)$$

Taylor Approximation — Results



Observations

- **1st-order** mean: $\hat{\mu} \approx f(\mu_z)$ (ignores curvature correction)
- **2nd-order** mean adds a bias correction proportional to the Hessian diagonal
- Variance quality depends on the nonlinearity of f

When to use

Best when f is smooth and the noise variances ν_i are small relative to the function's length scale.

Taylor Approximation in Python

```
1 import numpy as np
2
3 def taylor_approx(f, mu_z, nu_z, second_order=False, eps=1e-5):
4     """Estimate mean and variance via Taylor approximation."""
5     mu_z = np.asarray(mu_z, float); nu_z = np.asarray(nu_z, float)
6     n = len(mu_z); f0 = f(mu_z)
7     # Numerical gradient
8     grad = np.array([(f(mu_z+np.eye(n)[i]*eps)-f(mu_z-np.eye(n)[i]*eps))/(2*eps)
9                     for i in range(n)])
10    mu_hat = f0
11    nu_hat = np.dot(grad**2, nu_z)
12    if second_order:
13        hess_d = np.array([(f(mu_z+np.eye(n)[i]*eps)-2*f0+f(mu_z-np.eye(n)[i]*eps))/eps**2
14                            for i in range(n)])
15        mu_hat += 0.5 * np.dot(hess_d, nu_z)
16        nu_hat += 0.5 * np.dot(hess_d**2, nu_z**2)
17    return mu_hat, nu_hat
18
19 # Example 21.1:  $f(x, z) = \sin(x+z[0]) \cdot \cos(x+z[1])$ 
20 x = 0.5
21 f = lambda z: np.sin(x+z[0]) * np.cos(x+z[1])
22 mu_z, nu_z = [0.0, 0.0], [0.1, 0.2]
23 mu1, nu1 = taylor_approx(f, mu_z, nu_z, second_order=False)
24 mu2, nu2 = taylor_approx(f, mu_z, nu_z, second_order=True)
25 print(f"1st: mu={mu1:.4f}, nu={nu1:.4f}")
26 print(f"2nd: mu={mu2:.4f}, nu={nu2:.4f}")
```

21.3 Polynomial Chaos — Idea

Surrogate Model (Eq. 21.7)

Approximate $f(z)$ with k polynomial basis functions:

$$f(z) \approx \hat{f}(z) = \sum_{i=1}^k \theta_i b_i(z)$$

θ_i : coefficients to fit

$b_i(z)$: basis polynomials (degree $i - 1$, $b_1 = 1$)

Key insight

With *orthogonal* bases:

$$\hat{\mu} = \theta_1$$

$$\hat{\nu} = \sum_{i=2}^k \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z) dz$$

No numerical integration needed for $\hat{\mu}$!

Workflow:

- 1 Choose basis polynomials orthogonal under $p(z)$
- 2 Fit coefficients θ_i to function evaluations
- 3 Read off mean and variance analytically

Orthogonality and Mean/Variance Derivation

Orthogonality Condition (Eq. 21.20)

Two basis functions b_i and b_j are **orthogonal** w.r.t. $p(z)$ if:

$$\int_{\mathcal{Z}} b_i(z) b_j(z) p(z) dz = 0 \quad (i \neq j)$$

Mean (Eqs. 21.21–21.24)

With $b_1(z) = 1$ and all bases orthogonal:

$$\hat{\mu} = \theta_1 \int_{\mathcal{Z}} p(z) dz = \theta_1$$

(since all cross-terms vanish)

Variance (Eqs. 21.25–21.28)

$$\hat{v} = \sum_{i=2}^k \theta_i^2 \int_{\mathcal{Z}} b_i(z)^2 p(z) dz$$

Cross-terms vanish by orthogonality.
These integrals can be precomputed!

Orthogonal Basis Functions — Standard Families

Distribution	Domain	Density	Basis	Recurrence
Uniform	$[-1, 1]$	$\frac{1}{2}$	Legendre	$\frac{2i-3}{i-1} z b_{i-1} - \frac{i-2}{i-1} b_{i-2}$
Exponential	$[0, \infty)$	e^{-z}	Laguerre	$\frac{1}{i-1} ((2i-3-z)b_{i-1} - (i-2)b_{i-2})$
Unit Gaussian	$(-\infty, \infty)$	$\frac{1}{\sqrt{2\pi}} e^{-z^2/2}$	Hermite	$z b_{i-1}(z) - \frac{d}{dz} b_{i-1}(z)$

Table 21.1: Orthogonal polynomial basis functions

Three-term recurrence (Eq. 21.29):

with $b_1(z) = 1$ and weights:

$$b_{i+1}(z) = \begin{cases} (z - \alpha_i) b_i(z) & i = 1 \\ (z - \alpha_i) b_i(z) - \beta_i b_{i-1}(z) & i > 1 \end{cases}$$

$$\alpha_i = \frac{\int z b_i^2 p \, dz}{\int b_i^2 p \, dz}, \quad \beta_i = \frac{\int b_i^2 p \, dz}{\int b_{i-1}^2 p \, dz}$$

Orthogonal Basis Functions — Plots

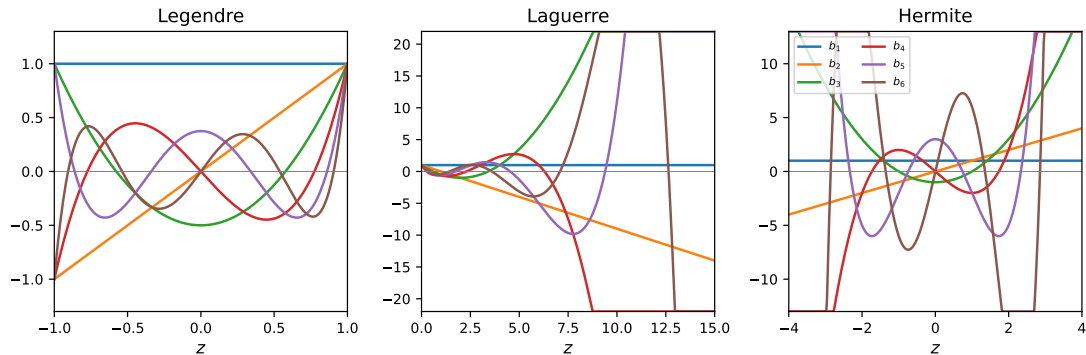


Figure 21.1: Legendre (uniform), Laguerre (exponential), Hermite (unit Gaussian) basis functions $b_1, \dots, b_6$.

21.3.2 Fitting the Coefficients

Method 1: Linear Regression

Given samples $z^{(1)}, \dots, z^{(m)}$ and values $f(z^{(i)})$, solve the least-squares problem:

$$\underbrace{\begin{bmatrix} b_1(z^{(1)}) & \cdots & b_k(z^{(1)}) \\ \vdots & \ddots & \vdots \\ b_1(z^{(m)}) & \cdots & b_k(z^{(m)}) \end{bmatrix}}_{\mathbf{B}} \underbrace{\begin{bmatrix} \theta_1 \\ \vdots \\ \theta_k \end{bmatrix}}_{\boldsymbol{\theta}} \approx \underbrace{\begin{bmatrix} f(z^{(1)}) \\ \vdots \\ f(z^{(m)}) \end{bmatrix}}_{\mathbf{y}}$$

Method 2: Gauss Quadrature (Eq. 21.38)

Exploit orthogonality to obtain the j -th coefficient directly:

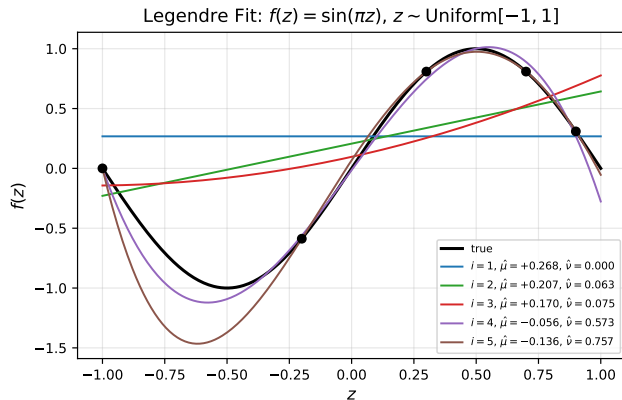
$$\theta_j = \frac{\int_{\mathcal{Z}} f(z) b_j(z) p(z) dz}{\int_{\mathcal{Z}} b_j(z)^2 p(z) dz}$$

The denominator is precomputed; the numerator is evaluated by Gaussian quadrature

Polynomial Chaos in Python — Legendre Fit

```
1 import numpy as np
2 from scipy.special import legendre as sp_legendre
3 from scipy import integrate
4
5 def poly_chaos_legendre(z_samples, f_samples, k):
6     """
7     Fit k Legendre basis functions and return (theta, mu_hat, nu_hat).
8     Assumes  $z \sim \text{Uniform}[-1,1]$ ,  $p(z) = 1/2$ .
9     """
10    # Build Vandermonde matrix  $B[i,j] = P_j(z_i)$ 
11    B = np.column_stack([sp_legendre(j)(z_samples) for j in range(k)])
12    # Least-squares fit
13    theta, _, _, _ = np.linalg.lstsq(B, f_samples, rcond=None)
14    mu_hat = theta[0] # = theta_1 since  $b_1 = P_0 = 1$ 
15    # Var:  $\sum_{j \geq 1} \theta_j^2 \int P_j^2(z) * (1/2) dz$ 
16    #  $\int_{-1}^1 P_j^2(z)/2 dz = 1/(2j+1)$ 
17    nu_hat = sum(theta[j]**2 / (2*j+1) for j in range(1, k))
18    return theta, mu_hat, nu_hat
19
20 # Example 21.3:  $f(z) = \sin(\pi * z)$ , 5 samples
21 z_s = np.array([-1.0, -0.2, 0.3, 0.7, 0.9])
22 y_s = np.sin(np.pi * z_s)
23
24 for k in range(1, 6):
25     _, mu, nu = poly_chaos_legendre(z_s, y_s, k)
26     print(f"k={k}: mu_hat={mu:+.3f}, nu_hat={nu:.3f}")
27 # True: mu=0, nu=0.5
```

Example 21.3 — Legendre Fit to $\sin(\pi z)$



Setup

$f(z) = \sin(\pi z)$, $z \sim \text{Unif}[-1, 1]$

True: $\mu = 0, \nu = 0.5$

Five samples: $\{-1, -0.2, 0.3, 0.7, 0.9\}$

Results by degree

i	$\hat{\mu}$	$\hat{\nu}$
1	+0.268	0.000
2	+0.207	0.063
3	+0.170	0.075
4	-0.056	0.573
5	-0.136	0.757

Higher degree $\rightarrow$ overfitting with only 5 points.

21.3.1 Stieltjes Algorithm — Arbitrary Distributions

Stieltjes Algorithm (Alg. 21.3)

Constructs orthogonal polynomial b_{i+1} from $\{b_1, \dots, b_i\}$ via the three-term recurrence, using **numerical integration**:

- 1 Compute $\alpha_i = \frac{\int_{\mathcal{Z}} z b_i(z)^2 p(z) dz}{\int_{\mathcal{Z}} b_i(z)^2 p(z) dz}$
- 2 If $i > 1$: compute $\beta_i = \frac{\int_{\mathcal{Z}} b_i(z)^2 p(z) dz}{\int_{\mathcal{Z}} b_{i-1}(z)^2 p(z) dz}$
- 3 Set $b_{i+1}(z) = (z - \alpha_i)b_i(z) - \beta_i b_{i-1}(z)$

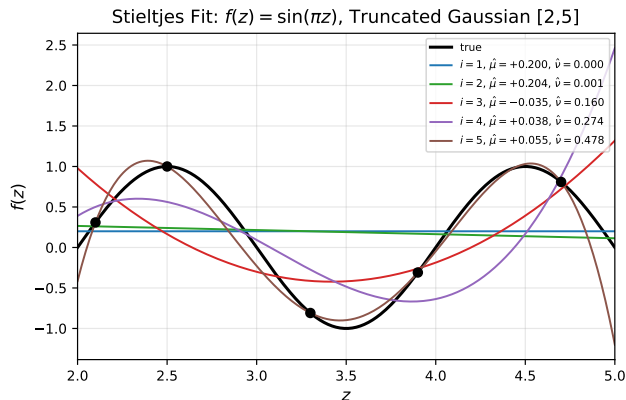
Why this matters

Works for **any** probability distribution with finite moments — not just the standard families (Legendre/Laguerre/Hermite). Essential when the input distribution is truncated, empirical, etc.

Stieltjes Algorithm in Python

```
1 import numpy as np
2 from scipy.stats import norm
3
4 def stieltjes_next(bs, p_func, zq):
5     """Build b_{i+1}: 3-term recurrence by numerical quadrature."""
6     pz = p_func(zq); bi = bs[-1]
7     alpha = np.trapz(zq*bi**2*pz,zq) / np.trapz(bi**2*pz,zq)
8     if len(bs) > 1:
9         bp = bs[-2]
10        beta = np.trapz(bi**2*pz,zq) / np.trapz(bp**2*pz,zq)
11        return (zq - alpha)*bi - beta*bp
12    return (zq - alpha)*bi
13
14 # Truncated Gaussian on [2,5]
15 a, b = 2.0, 5.0
16 nc = norm.cdf(b,3,1) - norm.cdf(a,3,1)
17 p = lambda z: norm.pdf(z,3,1)/nc
18 zq = np.linspace(a, b, 500)
19 bs = [np.ones_like(zq)]
20 for _ in range(4):
21     bs.append(stieltjes_next(bs, p, zq))
22 print(f"Constructed {len(bs)} orthogonal basis functions")
23 # Fit coefficients via linear regression on samples
24 z_s = np.array([2.1, 2.5, 3.3, 3.9, 4.7])
25 # Evaluate each basis at sample points...
```

Example 21.4 — Stieltjes for Truncated Gaussian



Setup

$$f(z) = \sin(\pi z)$$

$z \sim \mathcal{N}(3, 1)$ truncated to $[2, 5]$

True: $\mu \approx 0.104, \nu \approx 0.495$

Samples: $\{2.1, 2.5, 3.3, 3.9, 4.7\}$

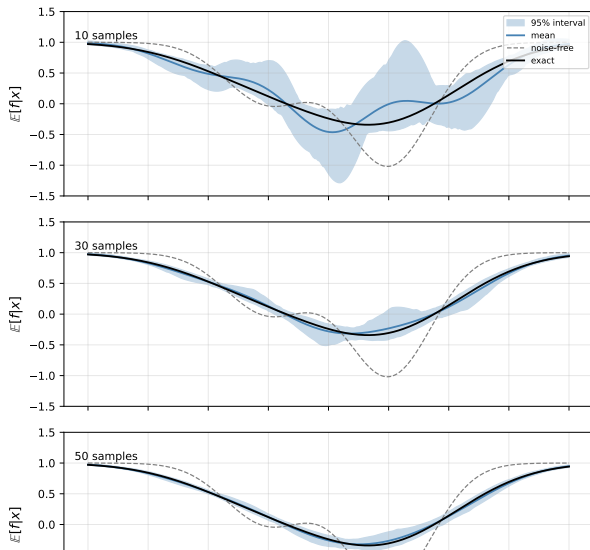
Results by degree

i	$\hat{\mu}$	$\hat{\nu}$
1	+0.200	-0.000
2	+0.204	0.001
3	-0.036	0.159
4	+0.037	0.273
5	+0.055	0.478

Higher degree captures more variance.

Example 21.2 — Hermite Chaos for Unknown Function

Polynomial Chaos (Hermite, 3rd order) — Example 21.2



Setup

$$f(x, z) = 1 - e^{-(x+z-1)^2} - 2e^{-(x+z-3)^2}$$

$z \sim \mathcal{N}(0, 1)$; 3rd-order Hermite polynomials

Observations

- 10 samples: wide uncertainty
- 30 samples: mean starts tracking true
- 50 samples: good agreement with true expected value

Blue band = 95% bootstrap interval over repeated fits.

21.3.3 Multivariate Polynomial Chaos

Multivariate Basis via Product Construction (Eq. 21.39)

For m random variables, multivariate basis functions are products of univariate bases:

$$b_i(\mathbf{z}) = \prod_{j=1}^m b_{a_j}(z_j)$$

where $\mathbf{a} = [a_1, \dots, a_m]$ is an **assignment vector** selecting the basis order for each dimension.

Example 21.5 — 3D basis

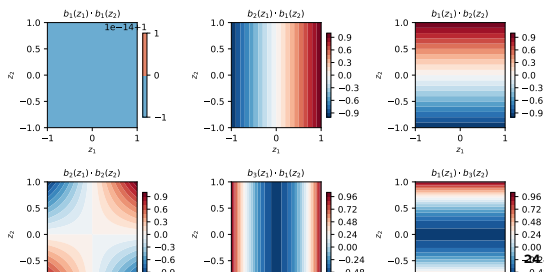
$$b(\mathbf{z}) = b_3(z_1) b_1(z_2) b_3(z_3)$$

assignment vector $\mathbf{a} = [3, 1, 3]$

The surrogate is still linear:

$f(\mathbf{z}) \approx \hat{f}(\mathbf{z}) = \sum_{i=1}^k \theta_i b_i(\mathbf{z})$ and mean/variance formulas carry over.

Multivariate Polynomial Chaos Basis Functions (Legendre)



Multivariate Polynomial Chaos in Python

```
1 import numpy as np
2 from itertools import product as iterproduct
3 from scipy.special import legendre as sp_legendre
4
5 def poly_chaos_bases_mv(bases_1d):
6     """
7     Build multivariate basis functions from 1-D bases.
8     bases_1d : list of lists; bases_1d[j] = [b1_j, b2_j, ...] for variable j
9     Returns  : list of callables, each mapping z (array) -> float
10    """
11    funcs = []
12    for combo in iterproduct(*bases_1d):
13        # combo = (b_{a_1}, b_{a_2}, ..., b_{a_m})
14        def make_basis(c):
15            return lambda z: np.prod([c[j](z[j]) for j in range(len(c))])
16        funcs.append(make_basis(combo))
17    return funcs
18
19 # Example: 2 variables, Legendre degree 0,1,2
20 bases_z1 = [sp_legendre(j) for j in range(3)] # b1,b2,b3
21 bases_z2 = [sp_legendre(j) for j in range(3)]
22 mv_bases = poly_chaos_bases_mv([bases_z1, bases_z2])
23 print(f"Number of multivariate bases: {len(mv_bases)}") # -> 9
24
25 # Evaluate at a point
26 z_test = np.array([0.5, -0.3])
27 vals = [b(z_test) for b in mv_bases]
```

21.4 Bayesian Monte Carlo

Idea

Use a **Gaussian process** (GP) — a distribution over functions — as a surrogate for f . Integrating over both the GP posterior and $p(\mathbf{z})$:

$$\mathbb{E}_{\mathbf{z} \sim p} [f] \approx \mathbb{E}_{\hat{f} \sim p(\hat{f})} \left[\mathbb{E}_{\mathbf{z} \sim p} [\hat{f}] \right] = \int_{\mathcal{Z}} \hat{\mu}(\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

where $\hat{\mu}(\mathbf{z})$ is the GP posterior mean (Eqs. 21.41–21.44).

Variance of the estimate

$$\text{Var}_{\mathbf{z} \sim p} [f] \approx \int_{\mathcal{Z}} \int_{\mathcal{Z}} \text{Cov}(\hat{f}(\mathbf{z}), \hat{f}(\mathbf{z}')) p(\mathbf{z}) p(\mathbf{z}') d\mathbf{z} d\mathbf{z}'$$

where $\text{Cov}(\hat{f}(\mathbf{z}), \hat{f}(\mathbf{z}')) = k(\mathbf{z}, \mathbf{z}') - k(\mathbf{z}, \mathbf{Z}) \mathbf{K}^{-1} k(\mathbf{Z}, \mathbf{z}')$

Also called

Bayes-Hermite Quadrature

Advantages:

- Encodes prior smoothness
- More efficient than MC for smooth f
- Analytic formulas when $\mathbf{z}$ is Gaussian

Bayesian MC — Gaussian Kernel, Gaussian Inputs

Gaussian Kernel (Eq. 21.50)

$$k(\mathbf{z}, \mathbf{z}') = \exp\left(-\frac{1}{2} \sum_{i=1}^n \frac{(z_i - z'_i)^2}{w_i^2}\right), \quad \mathbf{W} = \text{diag}[w_1^2, \dots, w_n^2]$$

Analytic Mean for $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$ (Eq. 21.51)

$$\begin{aligned} \mathbb{E}_{\mathbf{z} \sim p}[\mathbf{f}] &\approx \mathbf{q}^\top \mathbf{K}^{-1} \mathbf{y} \\ q_i &= |\mathbf{W}^{-1} \boldsymbol{\Sigma}_z + \mathbf{I}|^{-1/2} \exp\left(-\frac{1}{2} (\boldsymbol{\mu}_z - \mathbf{z}^{(i)})^\top (\boldsymbol{\Sigma}_z + \mathbf{W})^{-1} (\boldsymbol{\mu}_z - \mathbf{z}^{(i)})\right) \end{aligned}$$

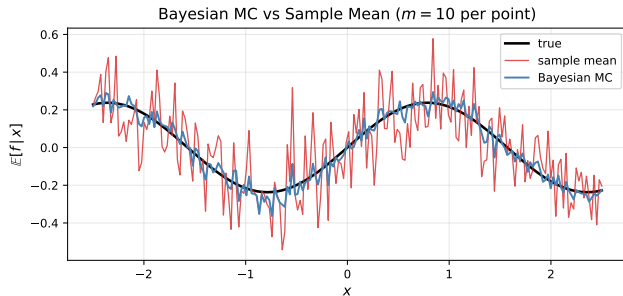
Analytic Variance (Eq. 21.53)

$$\text{Var}_{\mathbf{z} \sim p}[\mathbf{f}] \approx |\mathbf{W}^{-1} \boldsymbol{\Sigma}_z + \mathbf{I}|^{-1/2} - \mathbf{q}^\top \mathbf{K}^{-1} \mathbf{q}$$

Bayesian Monte Carlo in Python — Algorithm 21.5

```
1 import numpy as np
2
3 def gaussian_kernel(z1, z2, w):
4     diff = np.asarray(z1) - np.asarray(z2)
5     return np.exp(-0.5 * np.sum((diff / w)**2))
6
7 def build_K(Z, w):
8     m = len(Z)
9     return np.array([[gaussian_kernel(Z[i], Z[j], w)
10                        for j in range(m)] for i in range(m)])
11
12 def bayesian_monte_carlo(Z, y, w, mu_z, Sigma_z):
13     """
14     Algorithm 21.5: Bayesian Monte Carlo with Gaussian kernel.
15     Z          : (m, n) observed inputs
16     y          : (m,)  observed function values
17     w          : (n,)  kernel width parameters
18     mu_z       : (n,)  Gaussian mean of uncertainty
19     Sigma_z    : (n,n) Gaussian covariance of uncertainty
20     """
21     W = np.diag(w**2)
22     K = build_K(Z, w) + 1e-9 * np.eye(len(Z))
23     invK = np.linalg.inv(K)
24     # q vector
25     SzpW_inv = np.linalg.inv(Sigma_z + W)
26     det_fac = np.linalg.det(np.linalg.inv(W) @ Sigma_z + np.eye(len(w)))
27     q = np.array([np.exp(-0.5*(mu_z-z)@SzpW_inv@(mu_z-z)) for z in Z])
```

Example 21.6 — Bayesian MC vs Sample Mean



Setup

$$f(x, z) = \sin(x + z_1) \cos(x + z_2)$$

$$z_1 \sim \mathcal{N}(0, 1), z_2 \sim \mathcal{N}(0, 0.5)$$

$$\mu_z = [0, 0], \Sigma_z = \text{diag}[1, 0.5]$$

$$\text{Kernel weights } w = [1, 1]$$

$$m = 10 \text{ samples per } x$$

Key Result (at $x = 0$)

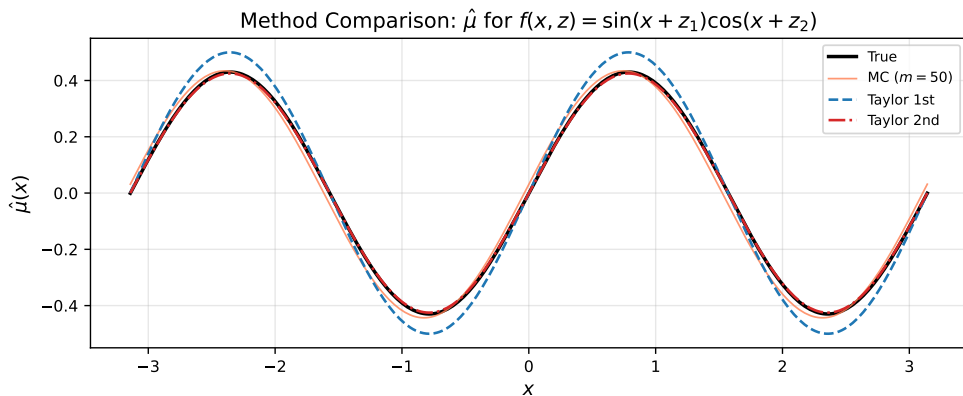
K matrix computed from 5 samples

$$q = [0.577, 0.450, 0.450, 0.417, 0.417]$$

$$\mathbb{E}[f] = 0.0, \text{Var}[f] = 0.327$$

Bayesian MC is **much smoother** than raw sample mean.

Method Comparison



Mean estimates for $f(x, z) = \sin(x + z_1)\cos(x + z_2)$

Methods at a Glance

Method	Assumptions	Pros	Cons
Monte Carlo	None	Simple; black-box	Slow convergence $\mathcal{O}(m^{-1/2})$
Taylor Approx.	f smooth, small ν_i	Analytic; no samples	Needs derivatives; poor for nonlinear f
Polynomial Chaos	f approx. polynomial	Efficient once bases fixed; analytic $\hat{\mu}, \hat{\nu}$	Exponential growth in variables
Bayesian MC	f smooth; GP prior	Very efficient; encodes prior	GP fitting overhead

21.5 Summary

- The **expected value** and **variance** of the objective function are useful when optimizing under uncertainty; computing them reliably can be challenging.
- **Monte Carlo integration** is the simplest approach: estimate moments by averaging function evaluations over random samples. Convergence rate is $\mathcal{O}(m^{-1/2})$.
- The **Taylor approximation** uses partial derivatives to analytically approximate the mean and variance; first- and second-order versions are available.
- **Polynomial chaos** fits an orthogonal polynomial surrogate to observed data; the mean equals the first coefficient θ_1 and the variance follows from the squared higher-order coefficients.
- **Bayesian Monte Carlo** uses Gaussian processes as surrogates; analytic formulas for Gaussian kernels and Gaussian inputs make it especially powerful.
- For **multivariate** inputs, polynomial chaos uses a product construction; the number of basis functions grows exponentially in the number of variables.

21.6 Exercises — Overview

Exercise 21.1

$z \sim \mathcal{N}(0, 1)$. Show: $P(|z| \leq 1) \approx 68.3\%$; $P(z \leq 1) \approx 84.1\%$. *Hint: use the CDF of the standard normal.*

Exercise 21.2

Let $x^{(1)}, \dots, x^{(m)}$ be i.i.d. with mean μ , variance ν . Show $\text{Var}(\hat{\mu}) = \nu/m$. *Key:* $\text{Var}(\sum a_i X_i) = \sum a_i^2 \text{Var}(X_i)$ for independent X_i .

Exercise 21.3

Derive the three-term recurrence relation (Eq. 21.29) from first principles. *Key: use the shift property* $\int (zb_j) b_j p \, dz = \int b_j (zb_j) p \, dz$.

Exercise 21.4

Derive an expression for $\partial\theta_j/\partial x_i$ using pseudo-inverse. Approximate: $[\partial\theta/\partial x_i] \approx B^+ [\partial f/\partial x_i]_{\text{at samples}}$.

Exercise 21.5 — Gradient of f_{mod} via Polynomial Chaos

Problem

The modified objective under uncertainty (Ch. 20): $f_{\text{mod}}(\mathbf{x}, \mathbf{z}) = \alpha \hat{\mu}(\mathbf{x}) + (1 - \alpha) \hat{\nu}(\mathbf{x})$
Derive $\partial f_{\text{mod}} / \partial x_i$ using polynomial chaos.

Solution

The chaos estimates satisfy:

$$\hat{\mu}(\mathbf{x}) = \theta_1(\mathbf{x}), \quad \hat{\nu}(\mathbf{x}) = \sum_{i=2}^k \theta_i^2(\mathbf{x}) \int_{\mathcal{Z}} b_i(z)^2 p(z) dz$$

Therefore:

$$\frac{\partial f_{\text{mod}}}{\partial x_i} = \alpha \frac{\partial \theta_1}{\partial x_i} + 2(1 - \alpha) \sum_{j=2}^k \theta_j(\mathbf{x}) \frac{\partial \theta_j}{\partial x_i} \int_{\mathcal{Z}} b_j(z)^2 p(z) dz$$

Coefficient gradients $\partial \theta_j / \partial x_i$ are estimated from Exercise 21.4 using the pseudo-inverse.

Full Python Example: All Methods Compared I

```
1 import numpy as np
2 from scipy.special import legendre as sp_legendre
3
4 # -- Setup -----
5 np.random.seed(42)
6 #  $f(x=0, z) = \sin(z_1)\cos(z_2)$ ,  $z_1 \sim N(0, 0.1)$ ,  $z_2 \sim N(0, 0.2)$ 
7 x = 0.0
8 def f(z): return np.sin(x + z[0]) * np.cos(x + z[1])
9 mu_z = np.array([0.0, 0.0])
10 nu_z = np.array([0.1, 0.2])
11
12 # -- Monte Carlo -----
13 m = 2000
14 Z = np.column_stack([np.random.randn(m)*np.sqrt(0.1),
15                      np.random.randn(m)*np.sqrt(0.2)])
16 fz = np.array([f(Z[i]) for i in range(m)])
17 mu_mc = np.mean(fz)
18 nu_mc = np.mean(fz**2) - mu_mc**2
19 print(f"Monte Carlo (m={m}): mu={mu_mc:.4f}, nu={nu_mc:.4f}")
20
21 # -- Taylor Approximation (1st order) -----
22 #  $df/dz_1|_{z=0} = \cos(x)^2$ ,  $df/dz_2|_{z=0} = -\sin(x)^2$ 
23 eps = 1e-5
24 e1 = np.array([eps, 0.0]); e2 = np.array([0.0, eps])
```

Full Python Example: All Methods Compared II

```
25 g1 = (f(mu_z+e1) - f(mu_z-e1)) / (2*eps)
26 g2 = (f(mu_z+e2) - f(mu_z-e2)) / (2*eps)
27 mu_ta = f(mu_z)
28 nu_ta = g1**2 * nu_z[0] + g2**2 * nu_z[1]
29 print(f"Taylor 1st order:      mu={mu_ta:.4f}, nu={nu_ta:.4f}")
30
31 # -- Polynomial Chaos (Legendre, univariate on z1 only) ----
32 # Marginalize: ignore z2 for illustration
33 z_s = np.linspace(-1, 1, 20)
34 y_s = np.array([f([z, 0.0]) for z in z_s])
35 k = 4
36 B = np.column_stack([sp_legendre(j)(z_s) for j in range(k)])
37 theta, _, _, _ = np.linalg.lstsq(B, y_s, rcond=None)
38 mu_pc = theta[0]
39 nu_pc = sum(theta[j]**2 / (2*j+1) for j in range(1, k))
40 print(f"Polynomial Chaos (k={k}): mu={mu_pc:.4f}, nu={nu_pc:.4f}")
```

Key Formulas at a Glance

Monte Carlo

$$\begin{aligned}\hat{\mu} &= \frac{1}{m} \sum f(\mathbf{z}^{(i)}) \\ \hat{\nu} &= \frac{1}{m} \sum f(\mathbf{z}^{(i)})^2 - \hat{\mu}^2 \\ \text{Var}[\hat{\mu}] &= \nu/m\end{aligned}$$

Polynomial Chaos (orthogonal)

$$\begin{aligned}f(\mathbf{z}) &\approx \sum_i \theta_i b_i(\mathbf{z}) \\ \hat{\mu} &= \theta_1 \\ \hat{\nu} &= \sum_{i=2}^k \theta_i^2 \int b_i^2 p \, d\mathbf{z}\end{aligned}$$

Taylor (1st order)

$$\begin{aligned}\hat{\mu} &= f(\boldsymbol{\mu}_z) \\ \hat{\nu} &= \sum_i (\partial f / \partial z_i)^2 \nu_i\end{aligned}$$

Bayesian MC (Gaussian)

$$\begin{aligned}\mathbb{E}[f] &\approx \mathbf{q}^\top \mathbf{K}^{-1} \mathbf{y} \\ \text{Var}[f] &\approx |2\mathbf{W}^{-1} \boldsymbol{\Sigma}_z + \mathbf{I}|^{-1/2} - \mathbf{q}^\top \mathbf{K}^{-1} \mathbf{q}\end{aligned}$$